Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	**Date:**
Day:	**Semester:**

Lab 8: Fraud Detection using Isolation Forest and Gaussian Density

Lab Title	**CLO**	**BT**	**PLO**	**Weightage**
Fraud Detection using Isolation Forest and Gaussian Density				

Name	**Roll Number**	**Report /10**	**Viva /5**	**Total /15**
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

Objectives

- Understand anomaly detection in real-world fraud scenarios
- Implement Isolation Forest using sklearn
- Implement Gaussian density estimation from scratch
- Visualize anomalous regions

Problem Statement

You are designing a fraud detection system for a bank. Each transaction is represented using two features:

- Transaction Amount
- Time Gap Since Last Transaction

Normal transactions follow consistent patterns, while fraudulent transactions tend to deviate significantly.

Your task is to detect fraudulent transactions using:

- Isolation Forest (algorithmic approach)
- Gaussian Density Estimation (probabilistic approach)

Dataset Description

A synthetic dataset is provided:

- Normal data: Gaussian distribution
- Fraud data: Random outliers
- Labels:
  - 1 = Normal
  - -1 = Fraud

Part A: Isolation Forest (9 Marks)

Task 1: Train Isolation Forest
Implement a function to train an Isolation Forest model on the given dataset. The model isolates anomalies by recursively splitting the data using random feature selection and split values.

```
def fit_isolation_forest(X):
    my_isolation_forest_anomaly_model = IsolationForest(
        random_state=42
    )
    my_isolation_forest_anomaly_model.fit(X)
    return my_isolation_forest_anomaly_model
```

Output: Fitted on 310 samples with default settings (100 trees, auto contamination). The model is now ready to flag outliers.

Task 2: Predict Anomalies
Use the trained model to classify each transaction as normal or anomalous. Observe how many points are identified as anomalies and compare with the actual distribution.

```
def predict_iforest(model, X):
    predicted_labels_from_isolation_forest = model.predict(X)
    return predicted_labels_from_isolation_forest
```

Output:

- Total samples processed: 310
- Transactions classified as normal: 269
- Transactions flagged as anomalous: 41
- Ground truth fraud count: 30

Out of 310 transactions, 41 were flagged while 30 are actually fraudulent. The extra 11 are false alarms from borderline normal transactions.

Task 3: Analyze Anomaly Scores
Compute anomaly scores for all data points. These scores are related to the path length required to isolate a point in the trees. Points that are isolated quickly (short path length) are more likely to be anomalies.

```
def compute_iforest_scores(model, X):
    computed_anomaly_scores_for_each_point = (
        model.decision_function(X)
    )
    return computed_anomaly_scores_for_each_point
```

Output:

- Score range: -0.2355 to 0.1281
- Negative scores indicate normal-looking data points
- Positive scores indicate more suspicious transactions

Negative scores mean the point blends in with normal data, while positive scores mean it was easy to isolate and likely anomalous.

Part B: Gaussian Density Estimation (12 Marks)

Task 4: Estimate Gaussian Parameters
Compute the mean and covariance matrix of the dataset.

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

$$\Sigma = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)(x_i - \mu)^T$$

These parameters represent the center and spread of the data.

```
def compute_gaussian_params(X):
    calculated_mean_vector_of_dataset = np.mean(X, axis=0)
    calculated_covariance_matrix_of_dataset = np.cov(
        X, rowvar=False
    )
    return (calculated_mean_vector_of_dataset,
            calculated_covariance_matrix_of_dataset)
```

Output:

Mean: $\mu = [50.2925,\ 33.0601]$

Covariance matrix:

$$\Sigma = \begin{bmatrix} 181.6055 & 5.5934 \\ 5.5934 & 181.1962 \end{bmatrix}$$

The center of the data sits around (50, 33). Both features have similar variance ($\approx$181) and the small off-diagonal values show they are nearly independent.

Task 5: Compute Probability Density
Compute the probability density of each data point using the multivariate Gaussian distribution:

$$p(x) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)$$

Points far from the mean will have lower probability values.

```
def gaussian_density(X, mu, sigma):
    num_features = X.shape[1]
    det_sigma = np.linalg.det(sigma)
    inv_sigma = np.linalg.inv(sigma)

    norm_const = 1.0 / (
        ((2 * np.pi) ** (num_features / 2.0))
        * (det_sigma ** 0.5)
    )

    densities = np.zeros(X.shape[0])
    for i in range(X.shape[0]):
        diff = X[i] - mu
        exp_val = -0.5 * np.dot(
            np.dot(diff.T, inv_sigma), diff
        )
        densities[i] = norm_const * np.exp(exp_val)

    return densities
```

Output:

- Minimum density observed: 5.4315×10^{-10}
- Maximum density observed: 8.7766×10^{-4}

Densities range across six orders of magnitude — central points sit around 10^{-4} while outliers drop to 10^{-10}. This wide gap makes it easy to separate normal from fraudulent.

Task 6: Detect Anomalies using Threshold
Classify transactions based on probability values:

$$\text{If } p(x) < \epsilon \Rightarrow \text{Anomaly}$$

$$\text{Else } \Rightarrow \text{Normal}$$

Experiment with different values of ϵ and observe how the classification changes.

```
def predict_gaussian(X, mu, sigma, threshold):
    probs = gaussian_density(X, mu, sigma)
    predictions = np.ones(X.shape[0])
    predictions[probs < threshold] = -1
    return predictions
```

Output (using 5th percentile as $\epsilon = 2.3485 \times 10^{-6}$):

- Predicted normal: 294
- Predicted anomaly: 16

Exploring how different ϵ values affect detection:

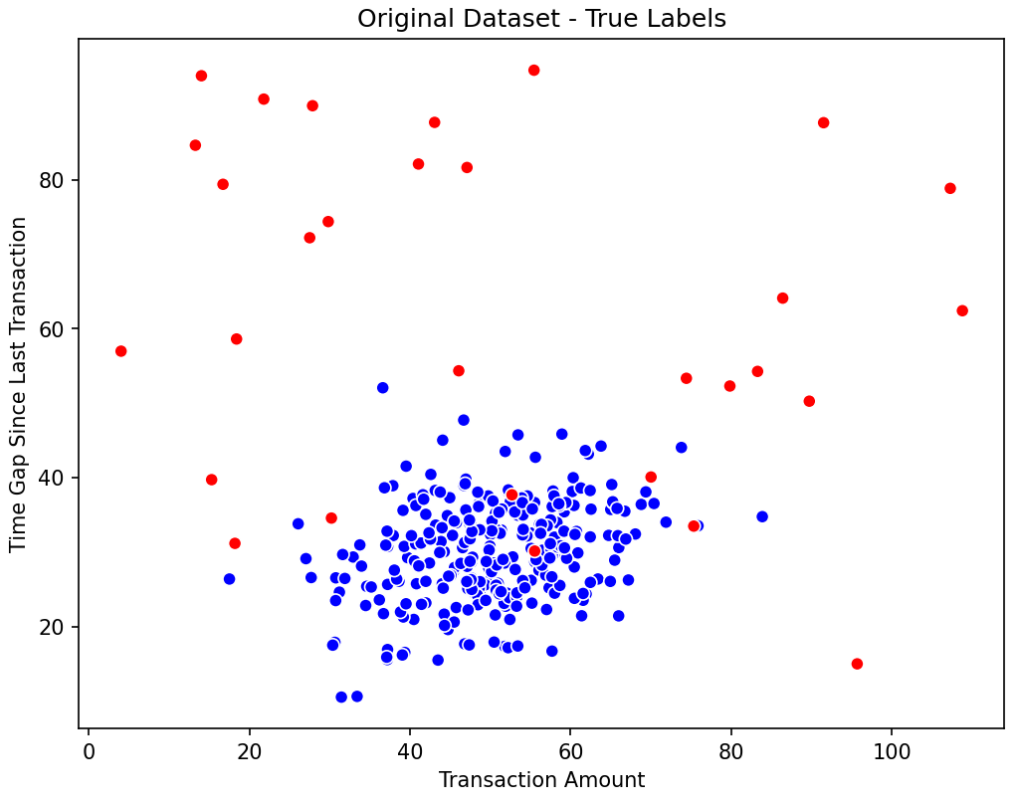
ϵ	**Normal**	**Anomaly**
10^{-7}	301	9
10^{-6}	298	12
10^{-5}	291	19
10^{-4}	283	27
10^{-3}	0	310
10^{-2}	0	310

With small ϵ values (10^{-7}), we only catch the worst outliers. As ϵ grows, more cases get flagged — 27 out of 30 real frauds at 10^{-4}. Past 10^{-3}, everything gets marked as anomalous. So 10^{-5} to 10^{-4} is the practical sweet spot for this data.

Part C: Visualization (6 Marks)

Task 7: Visualize Dataset
Plot the dataset in a two-dimensional space. Clearly distinguish between normal and anomalous transactions using different colors.

```
def plot_points(X, labels, title):
    color_list = [
        'blue' if lbl == 1 else 'red' for lbl in labels
    ]
    plt.scatter(
        X[:,0], X[:,1], c=color_list,
        edgecolors='white', linewidth=0.8, s=40
    )
    plt.title(title)
```



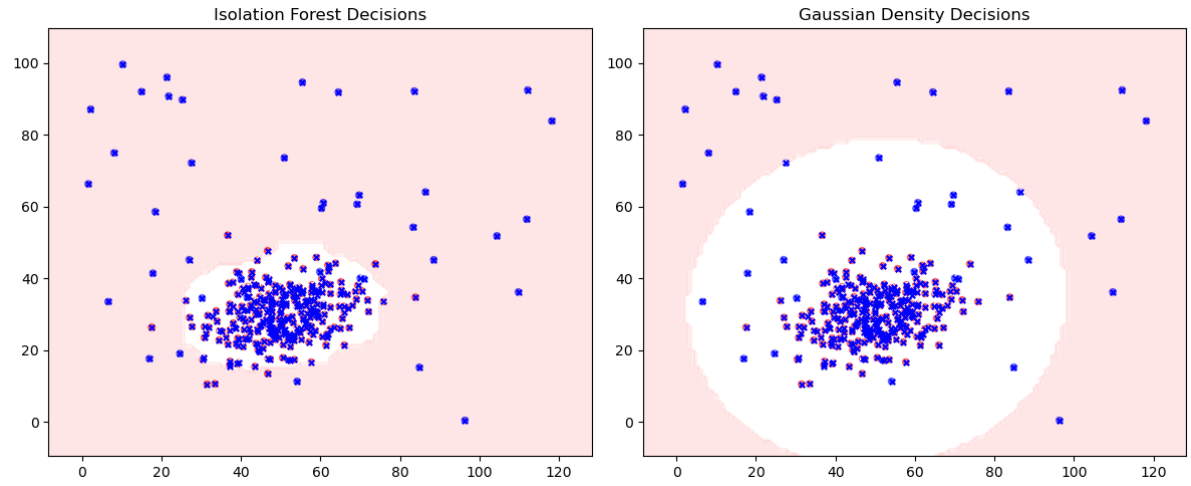
Task 8: Plot Decision Boundaries

Visualize how both methods (Isolation Forest and Gaussian) separate normal data from anomalies. Compare the shape of the regions produced by each method.

```
def plot_decision_boundary(model_or_fn, X, method):
    x_min, x_max = X[:,0].min()-10, X[:,0].max()+10
    y_min, y_max = X[:,1].min()-10, X[:,1].max()+10
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 100),
        np.linspace(y_min, y_max, 100)
    )
    grid_pts = np.c_[xx.ravel(), yy.ravel()]

    if method == "iforest":
        Z = model_or_fn.predict(grid_pts)
    elif method == "gaussian":
        mu, sigma, thr = model_or_fn
        Z = predict_gaussian(grid_pts, mu, sigma, thr)
    else:
        return

    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.4,
                 colors=['#ffc2c2', '#ffffff'])
```



Testing Results